Inteligencia artificial para proyectos de infraestructura ASIR

Unidad 6 · Proyecto Intermodular II

Paco Aldarias

7 de noviembre de 2026

Índice

Inteligencia artificial para proyectos de infraestructura ASIR **1**
1. Tipos de inteligencia artificial . . . 2
Según su capacidad . . . 2
Según cómo aprende o resuelve tareas . . . 2
Generativa, conversacional y agéntica . . . 2
IA en la nube y ejecución local . . . 2
2. Herramientas de IA y usos en ASIR . . . 3
Ejemplos de tareas adecuadas . . . 4
Límites y uso responsable . . . 4
3. Qué es el *vibecoding* . . . 5
4. Etapas: de la idea a GitHub . . . 5
1. Definir el problema y el alcance . . . 5
2. Recopilar requisitos y restricciones . . . 5
3. Diseñar antes de generar . . . 5
4. Preparar el espacio de trabajo . . . 5
5. Dar contexto e instrucciones concretas . . . 5
6. Construir en incrementos pequeños . . . 5
7. Validar funcionalidad y seguridad . . . 6
8. Documentar y preparar la publicación . . . 6
9. Publicar y mantener en GitHub . . . 6
5. Integrar varias IA en Visual Studio Code . . . 6
6. GitHub y la IA desde VS Code . . . 6
7. Cómo citar las fuentes y declarar el uso de IA . . . 7
Citas en el texto . . . 7
Cómo declarar una respuesta generada por IA . . . 7
8. Lista de comprobación para el proyecto . . . 7
9. Ejercicios prácticos de IA para un proyecto ASIR . . . 7
Ejercicio 1. Elegir entre una IA en la nube y una local . . . 7
Ejercicio 2. Consultar documentación con ayuda de IA . . . 8
Ejercicio 3. Diseñar y revisar un diagrama de red . . . 8
Ejercicio 4. Crear un script sencillo de comprobación . . . 8
Ejercicio 5. Investigar una incidencia con registros ficticios . . . 8
Ejercicio 6. Documentar el uso de IA en GitHub . . . 8
10. Bibliografía y fuentes consultadas . . . 9
Material docente . . . 9
Documentación oficial y páginas de producto . . . 9
Asistencia de IA utilizada en estos apuntes . . . 9

Volver al índice principal

Inteligencia artificial para proyectos de infraestructura ASIR

La inteligencia artificial (IA) reúne técnicas que permiten a los sistemas realizar tareas como reconocer patrones, generar contenido, clasificar información o proponer respuestas. En un proyecto de Administración

de Sistemas Informáticos en Red (ASIR), puede ayudar a investigar, diseñar, documentar y automatizar; no sustituye la comprensión técnica ni las pruebas.

> **Idea clave:** la IA propone; el equipo del proyecto revisa, prueba y asume la responsabilidad de lo que instala, configura y publica.

1. Tipos de inteligencia artificial

No existe una única clasificación: se puede hablar de IA según su capacidad, su método de aprendizaje o el tipo de tarea que realiza. Las categorías siguientes se solapan.

Según su capacidad

- **IA estrecha o especializada (ANI):** está diseñada para tareas concretas, como traducir, recomendar contenido, detectar anomalías o generar código. Es el tipo de IA que usamos hoy.
- **IA general (AGI):** sería capaz de aprender y resolver de forma flexible una gran variedad de tareas intelectuales, con una competencia comparable a la humana. No existe hoy como tecnología demostrada y disponible.
- **Superinteligencia artificial (ASI):** hipotética IA que superaría ampliamente las capacidades humanas en numerosos campos. Es una idea teórica, no un producto actual.

Según cómo aprende o resuelve tareas

- **IA simbólica o basada en reglas:** aplica reglas explícitas, lógica y conocimiento introducido por personas. Puede resultar comprensible y predecible, pero depende de que las reglas cubran los casos relevantes.
- **Aprendizaje automático (machine learning):** aprende patrones a partir de ejemplos. Puede ser supervisado (con ejemplos etiquetados), no supervisado (busca estructuras sin etiquetas) o por refuerzo (aprende mediante acciones y recompensas).
- **Aprendizaje profundo (deep learning):** utiliza redes neuronales con muchas capas para aprender patrones complejos, por ejemplo en imágenes, voz y lenguaje. Es una familia de técnicas de aprendizaje automático.
- **IA generativa:** crea contenido nuevo —texto, imágenes, audio o código— a partir de una petición y del contexto. Los modelos de lenguaje grandes (LLM) son un ejemplo. Pueden producir respuestas convincentes pero incorrectas; no son una fuente de verdad por sí solos.

Generativa, conversacional y agéntica

Estas categorías describen aspectos distintos y pueden combinarse en un mismo producto:

- **IA generativa:** crea contenido nuevo a partir de una instrucción: texto, imágenes, resúmenes o código. Por ejemplo, ChatGPT, Gemini y GitHub Copilot pueden generar contenido y código.
- **IA conversacional:** permite dialogar mediante texto o voz, recordar el contexto de la conversación y responder preguntas. ChatGPT y Gemini son ejemplos; una conversación puede usar IA generativa para redactar cada respuesta.
- **IA agéntica (o basada en agentes):** puede planificar una tarea en varios pasos y utilizar herramientas —por ejemplo, buscar archivos, modificar código o ejecutar pruebas— para intentar alcanzar un objetivo. Un agente de programación en VS Code es un ejemplo. Al concederle herramientas, también se le conceden capacidades de actuación: limita permisos, revisa las acciones y exige aprobación antes de operaciones delicadas.
- **IA de apoyo al estudio e investigación:** analiza fuentes que aporta el usuario para ayudar a resumir, comparar o preparar preguntas. NotebookLM es un ejemplo; comprueba que las respuestas estén respaldadas por las fuentes originales.

No son etiquetas excluyentes: una herramienta conversacional puede ser generativa y ofrecer funciones agénticas. El nombre comercial no garantiza que todas esas funciones estén activadas.

IA en la nube y ejecución local

La diferencia principal es dónde se procesa la petición. El modelo, el dispositivo, la conexión y la configuración concreta determinan las capacidades y el tratamiento de los datos. Estos ejemplos y esta distinción se apoyan en los materiales de UD6/anexos (Aldarias Raya, 2026).

Modalidad	Ejemplos de software	Ventajas	Aspectos que hay que valorar
En la nube: se accede a un servicio remoto por Internet.	ChatGPT, Gemini, NotebookLM y GitHub Copilot.	No suele requerir descargar y gestionar el modelo; permite utilizar servicios actualizados desde distintos dispositivos.	Requiere conexión y una cuenta; la petición puede salir del equipo. Revisa la política de privacidad, la configuración, las condiciones de la cuenta y las normas de la organización antes de enviar datos.
Local o en el propio equipo: un modelo se ejecuta en el ordenador o servidor gestionado por el usuario.	Ollama y LM Studio permiten obtener y ejecutar modelos compatibles localmente.	Permite trabajar sin enviar cada petición a un proveedor remoto y da más control sobre el entorno y los datos.	Se necesitan recursos de hardware, espacio, actualizaciones y conocimientos de administración. La calidad y velocidad varían según el modelo y el equipo. La instalación local no garantiza por sí sola la seguridad: el equipo, los registros, las extensiones y los modelos también deben protegerse.
Híbrida: se combinan componentes locales y remotos.	Un asistente local con acceso opcional a una API remota, según la configuración.	Permite elegir qué tareas o datos procesa cada componente.	Hay que entender qué información se envía, dónde se procesa y qué proveedor recibe cada petición.

Regla práctica: no asumas que una herramienta es local o privada solo por instalar una aplicación. Comprueba qué modelo está activo, si se hacen llamadas a la nube y qué datos se guardan o transmiten.

2. Herramientas de IA y usos en ASIR

Las herramientas cambian con rapidez y sus funciones dependen de la versión, el plan, la configuración y las políticas de cada organización. Los siguientes ejemplos ilustran usos habituales; comprueba las condiciones actuales antes de utilizarlos.

Tipo de herramienta	Ejemplos	Posibles usos en un proyecto ASIR
Asistentes conversacionales y generativos	**ChatGPT**, **Gemini**	Aclarar conceptos, comparar alternativas, resumir documentos, preparar listas de requisitos y proponer borradores de documentación o scripts. Comprueba las respuestas con fuentes fiables.
Asistente de programación	**GitHub Copilot** en VS Code	Obtener autocompletado, explicaciones y propuestas de código o pruebas; sus funciones de chat y agente dependen de la extensión, el plan y la configuración. Revisa y prueba los cambios.

Tipo de herramienta	Ejemplos	Posibles usos en un proyecto ASIR
Estudio e investigación con fuentes	**NotebookLM**	Consultar y resumir documentos aportados, preparar preguntas de repaso y localizar pasajes de apoyo. Confirma la respuesta en la fuente original y no cargues documentos confidenciales sin autorización.
Modelos locales	**Ollama**, **LM Studio**	Descargar y ejecutar modelos compatibles en un equipo o servidor propio para experimentar, resumir o generar borradores sin enviar cada petición a un servicio remoto. Verifica los requisitos y que el programa no utilice servicios en la nube para esa tarea.
Generación de imágenes y diagramas	Generadores de imágenes y herramientas de diagramación con IA	Crear una ilustración conceptual o un primer borrador de una arquitectura. Verificar que la topología y las etiquetas sean técnicamente correctas.
Herramientas de búsqueda y síntesis	Buscadores con respuestas asistidas por IA	Localizar documentación y obtener un resumen inicial. Contrastar las respuestas con documentación oficial y fecha de publicación.
Automatización y operaciones	Funciones inteligentes incluidas en plataformas de monitorización, nube o gestión	Resumir alertas o señalar patrones. Revisar siempre el diagnóstico antes de modificar sistemas.

En una infraestructura ASIR, pueden servir para comparar una solución de monitorización, entender un error de configuración, preparar un script de laboratorio o resumir documentación técnica. No les confíes contraseñas, cambios en producción ni la decisión final sobre la arquitectura.

Ejemplos de tareas adecuadas

- Pedir una explicación de las diferencias entre DNS recursivo y autoritativo, y verificarla en documentación fiable.
- Solicitar un borrador de inventario de servidores, una lista de requisitos o un esquema de secciones para la memoria técnica.
- Pedir que explique un script de Bash o PowerShell, o que sugiera casos de prueba para un script propio.
- Obtener un primer borrador de una configuración de laboratorio, para después validarlo en un entorno aislado.
- Resumir mensajes de error anonimizados y recibir hipótesis ordenadas para investigar.

Límites y uso responsable

- La IA puede inventar parámetros, comandos, opciones de software o referencias. Comprueba los detalles en documentación oficial.
- No pegues contraseñas, claves SSH, tokens, datos personales, direcciones sensibles ni configuraciones confidenciales en servicios externos.
- Elimina o sustituye nombres de host, dominios internos, direcciones IP públicas y cualquier dato que identifique una organización antes de compartir registros.
- No ejecutes comandos sugeridos directamente en producción. Léelos, entiende sus efectos y pruébalos en una máquina virtual o laboratorio.

- Revisa las condiciones de uso, la privacidad, la edad mínima, la cuenta y las opciones de retención de datos del servicio. En un equipo local, protege igualmente los archivos, registros y modelos.
- Concede a extensiones y agentes los permisos mínimos necesarios. Usa un entorno aislado; revisa los comandos y cambios propuestos y pide confirmación antes de instalaciones, borrados, cambios de permisos o acciones sobre sistemas reales.
- Contrasta hechos, versiones, comandos, citas y licencias con fuentes originales. No inventes referencias ni presentes como leída una fuente que no has consultado.
- Revisa derechos de autor, licencias y normas del centro o de la empresa antes de reutilizar código o contenido generado.
- Declara el uso de IA según las instrucciones del profesorado y conserva evidencias del proceso, las pruebas y las decisiones técnicas propias.

3. Qué es el *vibecoding*

Vibecoding (también escrito *vibe coding*) es una forma informal de crear software describiendo en lenguaje natural lo que se quiere y usando una IA para generar o modificar código mediante iteraciones. La persona dirige el trabajo con instrucciones y comentarios, mientras la IA propone cambios.

No significa que el proyecto se construya sin diseño ni revisión. En infraestructura, un fragmento incorrecto puede interrumpir servicios, abrir un acceso o borrar datos. Por eso, el *vibecoding* debe tratarse como una ayuda de desarrollo, no como una autorización para delegar decisiones o ejecutar cambios a ciegas.

4. Etapas: de la idea a GitHub

1. Definir el problema y el alcance

Describe qué necesidad resuelve el proyecto, quién lo usará y qué queda fuera. Por ejemplo: “montar un laboratorio reproducible con DNS, DHCP y monitorización para una red de pruebas”. Evita empezar con una petición vaga como “crea toda mi infraestructura”.

2. Recopilar requisitos y restricciones

Anota requisitos funcionales y no funcionales: servicios, usuarios, seguridad, disponibilidad, presupuesto, recursos, plazos y criterios de aceptación. Separa los datos confirmados de las suposiciones y no compartas secretos con el asistente.

3. Diseñar antes de generar

Decide la arquitectura y el entorno de pruebas: máquinas virtuales, contenedores o nube; topología; direcciones reservadas para el laboratorio; dependencias; copias de seguridad y controles de acceso. Pide a la IA alternativas y riesgos, pero justifica tú la decisión final.

4. Preparar el espacio de trabajo

Crea una carpeta y un repositorio Git. Añade un `README.md`, organiza los directorios y define cómo se probará el proyecto. Incluye un `.gitignore` adecuado para evitar publicar archivos temporales, secretos o estado local.

5. Dar contexto e instrucciones concretas

Indica al asistente el objetivo, el sistema operativo, las versiones relevantes, las restricciones y el resultado esperado. Pide cambios pequeños, archivos concretos y una explicación de cómo validarlos. No incluyas credenciales ni datos reales.

6. Construir en incrementos pequeños

Implementa una parte cada vez: primero una configuración mínima, después una función o servicio, y luego la siguiente integración. Tras cada paso, inspecciona el diff de Git, lee el código y ejecuta las pruebas en el laboratorio.

7. Validar funcionalidad y seguridad

Comprueba sintaxis, funcionamiento, permisos, puertos expuestos, manejo de errores, registros y recuperación. Usa validadores propios del formato cuando existan (por ejemplo, `terraform validate` para Terraform) y realiza pruebas de conectividad y de aceptación. No des por válida una respuesta solo porque "parece funcionar".

8. Documentar y preparar la publicación

Explica la arquitectura, los requisitos, la instalación, la configuración, las pruebas y la recuperación ante fallos. Sustituye secretos por variables de entorno o ejemplos ficticios. Revisa el historial y los archivos para confirmar que no se ha incluido ninguna clave.

9. Publicar y mantener en GitHub

Revisa el estado del repositorio y los cambios, crea commits con mensajes claros y publícalos en un repositorio de GitHub con visibilidad apropiada. Añade instrucciones de uso y licencia si corresponde. Si el trabajo continúa, usa ramas y revisiones mediante *pull requests*. Tras publicar, comprueba desde la web que los archivos y la documentación se ven correctamente.

> **Antes de publicar:** busca tokens, contraseñas, claves privadas, archivos `.env`, copias de seguridad y datos personales. Si se publica una credencial por error, borrarla del último commit no basta: revócala y rótala.

5. Integrar varias IA en Visual Studio Code

Visual Studio Code (VS Code) puede reunir el editor, GitHub y asistentes de IA mediante extensiones. La integración exacta varía; algunas extensiones usan la cuenta de su proveedor, otras una suscripción, una clave configurada por el usuario o un modelo local. Instala extensiones de editores y proveedores en los que confíes y confirma sus condiciones de privacidad.

1. **Instala VS Code y Git.** Comprueba que Git está disponible y abre la carpeta del proyecto como espacio de trabajo.
2. **Instala el asistente de GitHub.** Busca la extensión oficial de GitHub Copilot en la vista de extensiones, revisa el editor y los permisos, instálala e inicia sesión si tienes acceso. Sus prestaciones dependen del plan y la configuración.
3. **Añade otros proveedores si los necesitas.** Busca sus extensiones oficiales o mantenidas por el proveedor en el Marketplace. Sigue su documentación para iniciar sesión, configurar el modelo o conectar un servicio local.
4. **Usa cada asistente para una tarea acotada.** Por ejemplo, uno puede ayudarte a explicar un error y otro a revisar la documentación. Evita enviar el mismo código privado a varios servicios sin autorización.
5. **Revisa las respuestas en el propio proyecto.** Las sugerencias no son cambios verificados: inspecciona los archivos modificados, consulta el diff y ejecuta las pruebas.
6. **Comprueba la confianza del espacio de trabajo.** No habilites extensiones ni ejecutes tareas de una carpeta desconocida sin revisar sus permisos y configuración.

Tener varias extensiones instaladas no implica que colaboren entre sí ni que compartan contexto automáticamente. Pueden duplicar sugerencias, requerir cuentas distintas o enviar datos a servicios diferentes. Empieza con una herramienta, evalúa su utilidad y añade otra solo si aporta valor.

6. GitHub y la IA desde VS Code

Son piezas relacionadas, pero distintas:

- **Git** registra versiones y cambios del proyecto en el equipo.
- **GitHub** aloja repositorios Git y permite colaborar, revisar cambios y gestionar incidencias.
- **La extensión GitHub** facilita iniciar sesión y trabajar con repositorios o funciones de GitHub desde VS Code.
- **GitHub Copilot** es un asistente de IA integrado mediante extensiones; no es lo mismo que Git ni que GitHub.

Flujo habitual: abre una carpeta, inicializa o clona el repositorio Git, crea y revisa cambios, confirma los cambios (*commit*), inicia sesión en GitHub y publica o sincroniza el repositorio (*push*). Desde el editor se puede usar

un asistente para explicar o proponer código y, a la vez, utilizar el control de versiones para inspeccionar y guardar los cambios. Algunas funciones dependen de las extensiones y permisos instalados.

La IA no publica de forma responsable por sí sola: antes de enviar cambios, verifica que el repositorio no contenga secretos, información sensible ni configuraciones inseguras.

7. Cómo citar las fuentes y declarar el uso de IA

La IA es una herramienta de apoyo, no una fuente que sustituya a la documentación original. Para justificar un dato técnico, busca y cita el manual oficial, el artículo o el material docente del que se ha comprobado. Si una respuesta de IA indica una fuente, abre esa fuente y verifica que respalda realmente la afirmación.

Citas en el texto

Usa el estilo que solicite el profesorado. Si no se especifica uno, una cita autor-fecha sencilla permite identificar la fuente y localizarla en la bibliografía:

- Parafraseo: `La IA local puede dar más control sobre dónde se procesan los datos, pero necesita recursos del equipo (Aldarias Raya, 2026, p. 3).`
- Cita textual breve: `"la IA generativa crea contenido nuevo" (Aldarias Raya, 2026, p. 4).`
- Una página web sin fecha indicada: `(GitHub, s. f.)`; incluye el título y el enlace en la bibliografía.

Al citar literalmente, usa comillas y página si está disponible. Si resumes una idea, reescríbela con tus palabras sin cambiar su significado. Añade a la bibliografía toda fuente citada y evita atribuir a una fuente algo que no dice.

Cómo declarar una respuesta generada por IA

Sigue primero las reglas del módulo. Si permiten incluir contenido generado, identifícalo en el texto o en una nota metodológica, por ejemplo:

> Se utilizó [herramienta y modelo, si se conoce] el [fecha] para proponer un esquema y revisar la redacción de este apartado. Las afirmaciones técnicas se contrastaron con las fuentes citadas. La selección, comprobación y responsabilidad del resultado corresponden al autor.

Registra la herramienta y, si está disponible, el modelo o versión, la fecha, para qué se usó y qué partes del trabajo afectó. No incluyas prompts con información privada. Si el servicio permite compartir una conversación de forma segura y las normas lo requieren, añade el enlace; de lo contrario, describe el uso sin publicar datos sensibles. Las respuestas pueden cambiar y no siempre son recuperables, por eso no deben ser la única evidencia de un dato.

8. Lista de comprobación para el proyecto

- ☐ El problema, el alcance y los criterios de aceptación están escritos.
- ☐ Las decisiones de arquitectura se entienden y están justificadas.
- ☐ El código y las configuraciones sugeridas por IA se han revisado.
- ☐ Las pruebas se ejecutaron en un laboratorio aislado y sus resultados están documentados.
- ☐ No hay claves, datos personales ni información confidencial en los archivos o en el historial Git.
- ☐ El `README.md` explica cómo preparar, probar y detener el proyecto.
- ☐ El repositorio de GitHub tiene la visibilidad y los permisos adecuados.
- ☐ He declarado el uso de IA según las instrucciones de la asignatura y puedo explicar cada decisión técnica.

9. Ejercicios prácticos de IA para un proyecto ASIR

Realiza estos ejercicios con un proyecto de laboratorio o un caso ficticio. No introduzcas datos de una organización real en servicios externos, no ejecutes comandos generados por IA en producción y explica qué verificaste tú. En cada entrega, anota la herramienta utilizada, la fecha y cómo contrastaste el resultado.

Ejercicio 1. Elegir entre una IA en la nube y una local

1. Elige una tarea no confidencial del proyecto, por ejemplo, preparar un borrador de inventario de máquinas virtuales.

2. Compara una herramienta en la nube (ChatGPT o Gemini) con un modelo local ejecutado mediante Ollama o LM Studio.
3. Completa una tabla con conexión, requisitos de hardware, privacidad, facilidad de uso y limitaciones.
4. Justifica qué opción usarías para esa tarea y qué información no enviarías.

Entrega: tabla comparativa de una página y una conclusión breve. No afirmes que una opción es completamente privada sin comprobar su configuración.

Ejercicio 2. Consultar documentación con ayuda de IA

1. Selecciona una fuente pública y oficial relacionada con tu proyecto: por ejemplo, documentación de DNS, DHCP o un sistema operativo.
2. Pide a un asistente conversacional o a NotebookLM que resuma una sección o prepare cinco preguntas de repaso.
3. Comprueba cada respuesta en el documento original y anota su página, apartado o enlace.
4. Corrige cualquier respuesta imprecisa y añade la fuente a la bibliografía.

Entrega: resumen o cuestionario corregido, con al menos dos referencias comprobables. Usa únicamente documentos públicos o autorizados.

Ejercicio 3. Diseñar y revisar un diagrama de red

1. Escribe los requisitos de una red ficticia pequeña: una salida a Internet, dos subredes de laboratorio y servicios DNS y DHCP.
2. Pide a una IA que proponga un diagrama o una descripción textual de la topología.
3. Dibuja o adapta el diagrama con la herramienta que prefieras.
4. Revisa manualmente que las subredes no se solapen, que las puertas de enlace y los servicios estén identificados y que las conexiones tengan sentido.

Entrega: diagrama final y una lista de al menos tres revisiones o correcciones realizadas por ti. No conectes el diseño a una red real.

Ejercicio 4. Crear un script sencillo de comprobación

1. Pide a una IA un script de Bash o PowerShell que compruebe la resolución DNS de un nombre de ejemplo y si un host de laboratorio responde.
2. Solicita que explique cada línea y que no cambie la configuración ni elimine archivos.
3. Revisa el script y adapta el nombre de host a un entorno local controlado.
4. Ejecútalo en una máquina virtual o laboratorio y prueba tanto un resultado satisfactorio como uno de error.
5. Si algo falla, proporciona al asistente únicamente el mensaje de error anonimizado y analiza las sugerencias.

Entrega: script, explicación, resultados de las dos pruebas y una breve nota de los cambios que revisaste. No ejecutes código que no entiendas.

Ejercicio 5. Investigar una incidencia con registros ficticios

1. Prepara un fragmento ficticio de registro que represente un fallo de resolución DNS o una concesión DHCP incorrecta. No copies registros reales sin anonimizar.
2. Pide a la IA tres causas posibles, ordenadas por probabilidad, y una comprobación segura para cada causa.
3. Contrasta las comprobaciones con la documentación del servicio.
4. Indica qué evidencia permitiría confirmar o descartar cada hipótesis.

Entrega: tabla de hipótesis, pruebas y resultados. Deja claro que las sugerencias iniciales de la IA no son un diagnóstico confirmado.

Ejercicio 6. Documentar el uso de IA en GitHub

1. Usa una IA para preparar un primer borrador de una sección del `README.md` de un proyecto de laboratorio.
2. Comprueba instrucciones, requisitos, comandos y enlaces; corrige el borrador y cita las fuentes técnicas originales.

3. Incluye una nota breve que indique la herramienta usada, para qué se utilizó y qué verificaste.
4. Revisa con Git el diff y asegúrate de que no has añadido claves, tokens, datos personales ni configuraciones confidenciales.

Entrega: README revisado y declaración del uso de IA. No publiques el repositorio si contiene secretos; ante una credencial expuesta, rótala.

10. Bibliografía y fuentes consultadas

Material docente

- Aldarias Raya, F. (marzo de 2026). *Inteligencia Artifical* [material didáctico en PDF]. Tratamiento de la Información y Competencia Digital. Material de UD6, carpeta `anexos/TICD20-IA.pdf`.

Documentación oficial y páginas de producto

- GitHub. (s. f.). GitHub Copilot en VS Code. Consultado el 7 de noviembre de 2026.
- GitHub. (s. f.). Control de código fuente en VS Code. Consultado el 7 de noviembre de 2026.
- GitHub. (s. f.). Crear un repositorio en GitHub. Consultado el 7 de noviembre de 2026.
- Google. (s. f.). NotebookLM. Consultado el 7 de noviembre de 2026.
- Google. (s. f.). Gemini. Consultado el 7 de noviembre de 2026.
- LM Studio. (s. f.). Ejecutar modelos de lenguaje en LM Studio. Consultado el 7 de noviembre de 2026.
- Ollama. (s. f.). Modelos de IA en local y en la nube. Consultado el 7 de noviembre de 2026.
- OpenAI. (s. f.). ChatGPT. Consultado el 7 de noviembre de 2026.

Asistencia de IA utilizada en estos apuntes

Para organizar, redactar y revisar este material se utilizó un asistente de IA integrado mediante Copilot SDK en Visual Studio Code (modelo y proveedor subyacente no identificados en el entorno), el 7 de noviembre de 2026. Se usó como apoyo de redacción; no como fuente de hechos. La bibliografía técnica se basa en el material docente y los enlaces enumerados arriba. El contenido debe revisarse con el profesorado y las fuentes originales antes de reutilizarse.

Las interfaces, los nombres de funciones y las condiciones de los servicios pueden cambiar. Consulta la documentación oficial vigente y las normas de tu organización.

Descargar estos apuntes en PDF

Volver al índice principal